

Android Application Development

PSI / Gujarat Police Wireless Exam — સંપૂર્ણ ઊંડાણપૂર્વક સ્ટડી ગાઈડ
(Basic → Intermediate → Advanced → Exam-Level Calculations)

📖 સિવિલિયન પોઈન્ટ્સ

- આ ડોક્યુમેન્ટ બે સોર્સમાંથી બનાવવામાં આવ્યું છે: 'Android Application Development PSI Notes' (7 ટોપિક કવરેજ) અને 'Chapter-10 Export' (Strategic Overview, 70 Keyword Bank, Formula Sheet અને 40+ Practice MCQs).
- તમામ કોન્સેપ્ટ્સ, ટેબલ, કોડ, ફોર્મ્યુલા અને MCQ એક જ જગ્યાએ, ઊંડાણપૂર્વક ગુજરાતીમાં સમજાવવામાં આવ્યા છે.

📖 આવરી લેવાયેલા વિષયો:

- 1. Strategic Overview — Gujarat Police માટે Android શી માટે મહત્વનું છે + વેઈટેજ એનાલિસિસ
- 2. Android OS Concepts — Open Source, OHA, Versions, Core Components
- 3. Android Architecture — 5 Layers ઊંડાણપૂર્વક
- 4. Activities, Lifecycle અને Intent
- 5. UI Design — Layouts અને Widgets
- 6. Advanced UI Programming — Fragment, RecyclerView, Animation, Material Design
- 7. Toast, Menu, Dialog
- 8. List અને Adapters
- 9. SQLite Database — CRUD, Room, SQLCipher
- 10. ટેકનિકલ કીવર્ડ બેંક (70+ શબ્દો)
- 11. વર્તમાન પ્રવાહો — Gujarat Police Context (MDM, Biometric, 5G, Project Treble)
- 12. ફોર્મ્યુલા અને સિન્ટેક્સ શીટ (DPI Calculation સહિત)
- 13. પરીક્ષા ટિપ્સ (Mnemonics)
- 14. પ્રેક્ટિસ MCQ સેટ — સંપૂર્ણ સમજૂતી સાથે
- 15. ટૂંકા જવાબી પ્રશ્નો
- 16. માઈન્ડ મેપ રિક્રેપ

ભાગ ૧: સ્ટ્રેટેજિક ઓવરવ્યૂ — Android OS

Gujarat Police માટે Android કેમ મહત્વનું છે?

Gujarat Police ફિલ્ડ ઓપરેશન્સ માટે mobile computing પર ભારે આધાર રાખે છે. 'Pocket Cop' પ્રોજેક્ટ, E-Challan સિસ્ટમ, અને PCR વાનમાં mobile data terminals (MDTs) મોટાભાગે customized Android operating system પર ચાલે છે. Android architecture અને UI design સમજવું એ માત્ર syllabus પૂરતું નથી — તે ડ્યુટી પરના અધિકારીઓ દ્વારા વપરાતી secure communication અને data access apps ને deploy, troubleshoot અને maintain કરવાનો પાયો છે.

Part B વેઈટેજ એનાલિસિસ

પોઈન્ટ	વિગત
કુલ માર્ક્સ (Part B)	120 માર્ક્સ
IT & Communication Protocols ચંક	લગભગ 35-45 માર્ક્સ
Android-Specific વેઈટેજ	4 થી 7 MCQ (4-7 માર્ક્સ) અપેક્ષિત
ROI (Return on Investment)	ખૂબ ઊંચું — આ સેક્શનના પ્રશ્નો સામાન્ય રીતે factual/architectural હોય છે. Activity Lifecycle અને Architecture Layers માસ્ટર કરવાથી complex math વગર ગેરંટીડ માર્ક્સ મળે છે.

ભાગ ૨: Android OS Concepts

2.1 Android શું છે?

- Android એ Google દ્વારા develop કરાયેલ open-source, Linux-kernel based operating system છે, જે mainly mobile devices (smartphones, tablets) અને touchscreen devices માટે વપરાય છે.
- Android ને Open Handset Alliance (OHA) દ્વારા બનાવવામાં આવ્યું હતું.
- Android નું first version Android 1.0 હતું. શરૂઆતમાં દરેક version ને dessert name (Cupcake, Donut, Eclair...) આપવામાં આવતું, હવે number-based naming વપરાય છે.
- Android app Java, Kotlin અથવા C++ (NDK) માં લખાય છે. Android Studio એ official IDE (Integrated Development Environment) છે.

2.2 Android ની મુખ્ય લાક્ષણિકતાઓ (Key Features)

- Open Source (Free & Customizable)

- Linux Kernel Based
- Multi-tasking Support
- Rich UI Components
- Multiple Screen Sizes માટે Support
- Built-in Database Support (SQLite)
- Connectivity: Wi-Fi, Bluetooth, NFC, GPS
- Reuse અને Replacement ને Support કરતું Application Framework

2.3 Android Versions (MCQ માટે મહત્વનું)

Version Name	API Level
Cupcake	3
Donut	4
Eclair	5-7
Froyo	8
Gingerbread	9-10
Honeycomb	11-13
Ice Cream Sandwich	14-15
Jelly Bean	16-18
KitKat	19
Lollipop	21-22
Marshmallow	23
Nougat	24-25
Oreo	26-27
Pie	28
Android 10-15	29-35

2.4 Core Components — દરેક Android App ના ચાર પાયાના Building Blocks

- Activities: UI અને user interaction ને નિયંત્રિત કરે.
- Services: Background processing કરે (દા.ત., file download).
- Broadcast Receivers: System-wide events (દા.ત., low battery, SMS received) સાંભળે.
- Content Providers: Apps વચ્ચે shared data manage કરે (દા.ત., contact list access).

🎯 Elimination Strategy

- જો પ્રશ્નમાં 'app components' વિષે પુછાય, તો યાદ રાખો કે એ ચોક્કસ ચાર જ છે (Activity, Service, Broadcast Receiver, Content Provider). બીજું કંઈપણ (જેમ કે Intent કે Fragment) એ distractor છે.

2.5 Compilation Process

Java/Kotlin Code → Java Bytecode (.class) → Dalvik Executable (.dex) → Android Application Package (.apk)

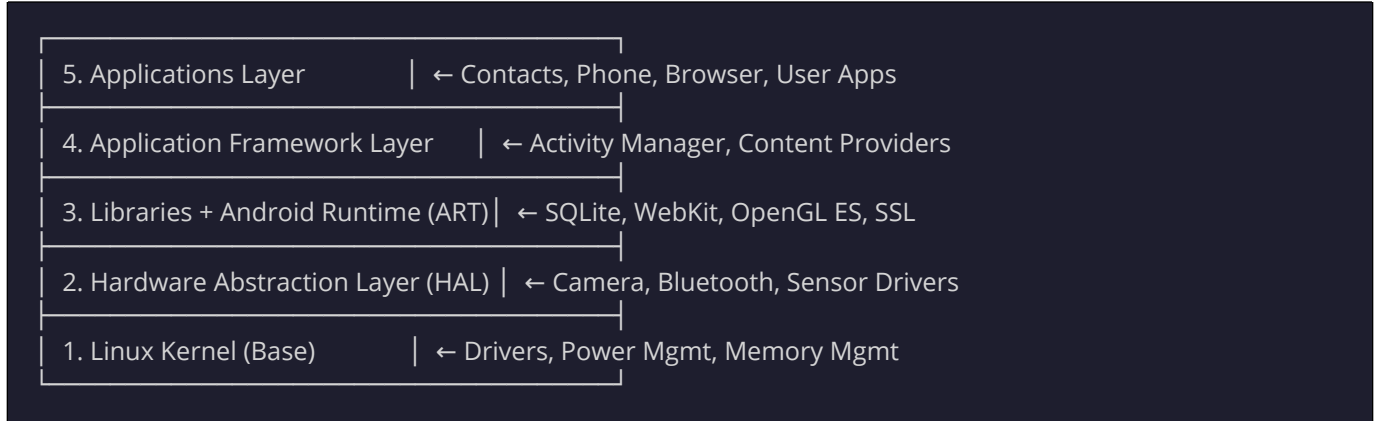
📝 સિવિલિયન પોઇન્ટ્સ

- Android એ mobile OS + middleware + key applications નું stack છે.
- Android app Java, Kotlin અથવા C++ (NDK) માં લખાય છે.
- Android Studio એ official IDE છે.

ભાગ 3: Android Architecture

Android Architecture એક software stack છે જે 5 અલગ layers માં વહેંચાયેલ છે (bottom to top). આ ટોપિક Android exam MCQ માટે #1 most tested topic પૈકીનું એક છે.

Layer Diagram (Bottom → Top)



3.1 Linux Kernel (Base Layer)

- સૌથી નીચેનું layer (base layer).
- Core system services પૂરી પાડે: security, memory management, process management, network stack, અને driver model (Display, Camera, Bluetooth વગેરે).
- Device drivers, power management પણ આ layer manage કરે છે.

3.2 Hardware Abstraction Layer (HAL)

- Standard interfaces પૂરા પાડે છે જે device hardware capabilities ને higher-level Java API framework સામે expose કરે.
- Hardware components (Camera, Bluetooth, Audio, Sensors) માટે standard interface પૂરો પાડે છે — જેથી Android framework, driver implementations થી agnostic રહી શકે.

3.3 Android Runtime (ART) & Native Libraries

- ART: Dalvik ને Android 5.0 થી replace કર્યું. Ahead-of-Time (AOT) અને Just-in-Time (JIT) compilation વાપરી performance સુધારે — installation સમયે bytecode ને native machine code માં compile કરી app launch speed અને battery performance સુધારે છે.
- Dalvik: જૂના Android versions (pre-5.0) માં વપરાતું discontinued process virtual machine.
- Native C/C++ Libraries: WebKit (browser engine), OpenGL ES/Vulkan (2D/3D ગ્રાફિક્સ), SQLite (database), SSL નો સમાવેશ થાય છે.

3.4 Application Framework Layer

- Developers ને API પૂરી પાડે છે જેનાથી apps બનાવાય (View System, Resource Manager, Notification Manager, Activity Manager).
- Activity Manager — Activities ના lifecycle ને manage કરે (Java API Framework નો ભાગ, System નહિ).
- Content Providers — Apps વચ્ચે data sharing.
- View System — UI components (buttons, lists વગેરે).
- Notification Manager — Notifications display કરે.
- Package Manager — App installation info manage કરે.

3.5 Applications Layer (Top Layer)

- સૌથી ઉપરનું layer — default apps જેમ કે Dialer, Home Screen, Email, Calendar, Camera, Browser, અને developer દ્વારા બનાવેલ third-party apps.

મહત્વના Architecture Components — ઝડપી સંદર્ભ

Component	કયા Layer / શું કામ
Binder Driver	Linux Kernel — Inter-Process Communication (IPC) સૌથી નીચલા સ્તરે handle કરે
Surface Manager	અલગ-અલગ apps ના 2D/3D graphic layers ને composite કરી એક જ frame તરીકે screen પર display કરે
Garbage Collector (GC)	ART/Runtime — વપરાયા વગરના objects reclaim કરે; Memory Leak ત્યારે થાય જ્યારે GC objects reclaim ન કરી શકે
WebKit	Native Library — WebView class ને power આપતું open-source browser engine
OpenGL ES / Vulkan	Native Library — 2D અને 3D ગ્રાફિક્સ rendering
SQLite	Native Library — embedded, lightweight relational database

🌀 Mnemonic — Android Architecture Layers (Bottom to Top)

- "Little Hardware Requires Fast Apps"
- L → Linux Kernel
- H → Hardware Abstraction Layer
- R → Runtime / Libraries
- F → Framework (Java API)
- A → Apps

📖 સિવિઝન પોઇન્ટ્સ

- Architecture bottom-to-top: Linux Kernel → HAL → Libraries+ART → Application Framework → Applications.

- ART, Dalvik VM ને replace કરે (Android 5.0 Lollipop થી default).
- SQLite database library Kernel layer નો ભાગ નથી — તે Libraries layer માં આવે છે.

ભાગ ૪: Android Activities, Lifecycle અને Intent

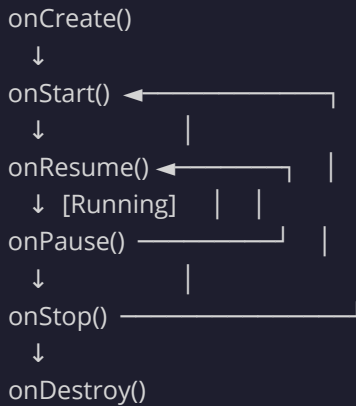
4.1 Activity શું છે?

- Activity એ single screen (UI) represent કરે છે જેની સાથે user interact કરી શકે.
- દરેક Activity AppCompatActivity અથવા Activity class ને extend કરે છે.
- App ના AndroidManifest.xml file માં દરેક Activity declare કરવી ફરજિયાત છે.

```
<activity android:name=".MainActivity">
  <intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
  </intent-filter>
</activity>
```

4.2 Activity Lifecycle ★ (પરીક્ષા માટે અત્યંત મહત્વનું)

Activity lifecycle Android exam માં #1 most tested topic છે. નીચેની flow અને table ધ્યાનથી યાદ રાખો:



(Stop થયા પછી ફરી શરૂ થાય તો → onStart() → onResume())

Method	ક્યારે Call થાય
onCreate()	Activity પ્રથમ વખત બને ત્યારે (UI અહીં setup થાય — setContentView())
onStart()	Activity user ને visible બને ત્યારે
onResume()	Activity foreground માં આવે અને user interact કરી શકે ત્યારે
onPause()	બીજી Activity partially cover કરે ત્યારે (દા.ત., transparent dialog) — data અહીં save કરવું જોઈએ
onStop()	Activity સંપૂર્ણપણે hidden/no longer visible થાય ત્યારે

Method	ક્યારે Call થાય
onRestart()	Stop થયા પછી ફરી start થાય તે પહેલાં
onDestroy()	Activity memory માંથી સંપૂર્ણપણે remove થાય ત્યારે
🔗 Mnemonic — Activity Lifecycle • "Cops Start Running Past Suspect's Door" • Create → Start → Resume → Pause → Stop → Destroy	
🔗 Golden Rule of Lifecycles (Exam Hacker Tip) • onPause() અને onStop() વચ્ચે ક્યારેય confuse ન થાય. જો user હજુ app નો ભાગ 'જોઈ' શકે પણ interact ન કરી શકે, તો એ Paused છે. જો બિલકુલ 'જોઈ' ન શકે, તો એ Stopped છે.	
Scenario	કયો Method Call થાય
User ને phone call આવે, Activity background માં જાય પણ destroy ન થાય	onPause() (ત્યારબાદ સંપૂર્ણ cover થાય તો onStop())
Screen rotate થાય	Activity destroy → recreate (scroll position/data બચાવવા onSaveInstanceState() વાપરો, જે Bundle માં transient UI state store કરે)
User Back button દબાવી app બંધ કરે	onPause() → onStop() → onDestroy()

4.3 Intent

- Intent એ message-passing/messaging object છે જે components વચ્ચે (અથવા બીજી app ના component પાસેથી action request કરવા) communication કરવા વપરાય છે.
- Explicit Intent — ચોક્કસ Activity/component ને class name દ્વારા target કરે.
- Implicit Intent — ચોક્કસ action (દા.ત. 'View Map', 'Send Email') specify કરે, અને કઈ app એ handle કરશે તે નક્કી ન કરે — system યોગ્ય component શોધે.
- Intent Filter — Manifest માં એક activity/service/broadcast receiver ની capabilities જાહેર કરે.
- બે Activities વચ્ચે data pass કરવા Bundle (Intent દ્વારા, putExtra() મેથડથી) વપરાય.

```
// Explicit Intent
Intent intent = new Intent(MainActivity.this, SecondActivity.class);
startActivity(intent);

// Implicit Intent
Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse("https://gujaratpolice.gujarat.gov.in"));
startActivity(intent);
```

4.4 Task Back-Stack અને Launch Modes

- Activity Manager (system service) task stack ને manage કરે (LIFO — Last In, First Out).
- FLAG_ACTIVITY_NEW_TASK અને FLAG_ACTIVITY_CLEAR_TOP — ઘણીવાર સાથે વાપરી back stack સાફ કરી નવું task શરૂ કરાય (દા.ત. Logout પછી).
- singleInstance — સૌથી restrictive launch mode; Activity પોતાના અલગ task માં run થાય, અને એ જ task માં બીજી કોઈ activity launch ન થઈ શકે.

4.5 ANR (Application Not Responding)

- Main UI thread > 5 seconds માટે block થાય (દા.ત. લાંબુ network request run કરવાથી) ત્યારે ANR trigger થાય.
- Main Thread UI updates માટે responsible છે — તેને block કરવાથી app freeze થાય.

રિવિઝન પોઈન્ટ્સ

- Activity lifecycle ના 7 methods યાદ રાખો (exam માં sequence પુછાય).
- Intent ના 2 types — Explicit અને Implicit.
- setContentView() method activity ને XML layout સાથે connect કરે.
- findViewById() — XML view ને Java/Kotlin code માં reference આપવા વપરાય.
- onSaveInstanceState() Bundle માં transient state store કરે, activity kill થતાં પહેલાં.

ભાગ ૫: UI Design — Layouts અને Widgets

5.1 XML આધારિત UI

- XML (eXtensible Markup Language) — Android માં UI layouts design કરવા વપરાય (res/layout ફોલ્ડરમાં).
- View — UI components જે basic building block (દા.ત. TextView, Button, ImageView).
- ViewGroup — બહુવિધ Views ને hold કરતું container. Layouts તરીકે પણ ઓળખાય.
- ViewTree — એક layout ના Views અને ViewGroups નું hierarchical structure.

5.2 Layout Types

Layout Type	ઉપયોગ
LinearLayout	Elements ને એક row (horizontal) અથવા column (vertical) માં ક્રમમાં ગોઠવે
RelativeLayout	Elements ને એકબીજાની અથવા parent ની relative position માં ગોઠવે
ConstraintLayout	Flat hierarchy; complex, flexible UI માટે highly optimized — આધુનિક ધોરણ (modern standard)
FrameLayout	Single element/overlap માટે વપરાય
TableLayout	Row-column (grid) structure માટે

```
<LinearLayout
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  android:orientation="vertical">
  <TextView
    android:id="@+id/tvName"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World" />
  <Button
    android:id="@+id/btnClick"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Click Me" />
</LinearLayout>
```

5.3 Basic UI Widgets

- TextView — Text display માટે.

- EditText — User input લેવા માટે.
- Button / ImageButton — Click action માટે.
- ImageView — Image display માટે.
- CheckBox / RadioButton — Selection માટે.

5.4 UI Design ટેકનિકલ Units — dp, sp અને Touch Target

- dp / dip (Density-independent Pixel): UI dimensions માટે standard unit, જેથી અલગ-અલગ screen densities પર uniform size જળવાય રહે.
- sp (Scale-independent Pixels): Text sizes માટે વપરાતું unit, જે user ના system font size settings ને respect કરે.
- Standard Minimum Touch Target Size: 48dp x 48dp — usability ensure કરવા highly tested UI design constraint.

5.5 dp થી px કન્વર્ઝન ફોર્મ્યુલા

$$px = dp \times (dpi / 160)$$

(યાદ રાખો: 160 dpi પર, 1 dp બરાબર 1 px)

📝 સિવિલિયન પોઈન્ટ્સ

- R.java — auto-generated file જેમાં layouts, strings, images જેવા તમામ resources ના Resource IDs હોય છે (build process દ્વારા generate થાય).
- 48dp x 48dp — minimum touch target size, MCQ માટે highly tested.
- ConstraintLayout — flat hierarchy, complex UI માટે performance-efficient, આધુનિક standard.

ભાગ ૬: Advanced UI Programming

6.1 Fragment

- Fragment એ Activity ની અંદર reusable, modular UI portion છે.
- Fragment નું પોતાનું lifecycle હોય છે, પણ તે parent Activity ના lifecycle સાથે જોડાયેલું હોય છે.
- Multi-pane UI (tablets) માટે ખૂબ ઉપયોગી.
- Fragment નું UI Activity ની અંદર embed થાય — independent screen નહિ.

```
FragmentManager fm = getSupportFragmentManager();
FragmentManager ft = fm.beginTransaction();
ft.replace(R.id.fragment_container, new MyFragment());
ft.commit();
```

6.2 RecyclerView (Advanced List)

- ListView કરતાં વધુ efficient, flexible, modern list component.
- ViewHolder pattern ફરજિયાત વાપરે — scroll off-screen થતાં views ને recycle કરે, જેનાથી memory ખૂબ બચે અને performance સુધરે.
- LayoutManager (LinearLayoutManager, GridLayoutManager, StaggeredGridLayoutManager) જરૂરી છે.
- LinearLayoutManager — standard vertical list બનાવવા default/જરૂરી.

```
RecyclerView recyclerView = findViewById(R.id.recyclerView);
recyclerView.setLayoutManager(new LinearLayoutManager(this));
recyclerView.setAdapter(new MyAdapter(dataList));
```

6.3 Custom Views

- Developer પોતાની જરૂરિયાત મુજબ View class extend કરી custom UI component બનાવી શકે (દા.ત. custom clock, graph).

6.4 Animations

- Tween Animation — View ના property (position, size, rotation) પર animate થાય.
- Property Animation (ObjectAnimator) — Object ના actual property ને animate કરે (વધુ powerful).
- Frame Animation — Images ની sequence બતાવે (movie-like).

6.5 Styles & Themes

- Style — Single View માટે attributes નો set (res/values/styles.xml).
- Theme — સંપૂર્ણ Activity/App પર apply થતો style.

6.6 Material Design

- Google દ્વારા આપેલ design language — depth, motion, responsive UI પર focus.
- Components: CardView, FloatingActionButton (FAB), NavigationDrawer, BottomNavigationView, Snackbar.
- Snackbar, Toast કરતાં preferred છે કારણ કે તેમાં interactive action button (દા.ત. 'UNDO') સમાવી શકાય — Toast ફક્ત informational હોય છે.

રિવિઝન પોઇન્ટ્સ

- Fragment vs Activity: Fragment જું UI Activity ની અંદર embed થાય, independent screen નહિ.
- RecyclerView, ListView કરતાં વધુ efficient (ViewHolder pattern ના કારણે).
- ListView legacy component ગણાય છે; RecyclerView modern standard છે.
- Material Design components exam માં naming-based MCQ માટે યાદ રાખો.

ભાગ ૭: Toast, Menu, Dialog

7.1 Toast

- Toast એ short, non-blocking feedback message છે જે screen પર થોડી second માટે નાના overlay તરીકે દેખાય અને automatically અદ્રશ્ય થાય.
- તે current Activity પાસેથી focus 'ચોરતું' નથી — user એની સાથે interact કરી શકતો નથી.
- Duration constants: Toast.LENGTH_SHORT (~2 sec) અથવા Toast.LENGTH_LONG (~3.5 sec).

```
Toast.makeText(getApplicationContext(), "Hello Toast!", Toast.LENGTH_SHORT).show();
```

🔗 The Toast Trap (Exam Hacker Tip)

- Toast ક્યારેય screen પર દેખાશે નહિ જ્યાં સુધી syntax ના અંતે .show() explicitly call ન કરવામાં આવે.
- જો MCQ પૂછે 'આ code શું કરશે?' અને .show() missing હોય, તો સાચો જવાબ 'Nothing happens' છે — examiners ઈરાદાપૂર્વક આ ટ્રેપ મૂકે છે.

7.2 Menu — ત્રણ પ્રકાર

Menu Type	વર્ણન
Options Menu	App bar/toolbar (top-right) માં દેખાતું primary menu — activity ના menu items નું મુખ્ય collection
Context Menu	કોઈ element પર long-click કરવાથી ખૂલતું floating menu (item-specific actions માટે ઉપયોગી, દા.ત. lists)
Popup Menu	View ની નીચે anchor થઈને દેખાતું items ની list

- Context Menu માટે registerForContextMenu(view) લાઈન onCreate() માં ફરજિયાત જોઈએ — view ને explicitly register કરવો પડે.
- onOptionsItemSelected(Menuitem item) — Options Menu માં item select થાય ત્યારે call થતું standard override.
- res/menu ડિરેક્ટરી — menu XML resources (દા.ત. main_menu.xml) ડિફાઈન કરવા standard location.

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(R.menu.main_menu, menu);
    return true;
}

@Override
public boolean onOptionsItemSelected(Menuitem item) {
    switch (item.getItemId()) {
```

```

case R.id.action_settings:
    // Settings action
    return true;
}
return super.onOptionsItemSelected(item);
}

```

7.3 Dialog

- Dialog એ small window છે જે current Activity ની ઉપર દેખાય, user ને decision લેવા/information આપવા/extra input માંગવા માટે.
- AlertDialog — Message + Positive/Negative/Neutral buttons (Yes/No જેવા) સાથે. Positive = accept/continue ('OK'/'Delete'), Negative = cancel ('Cancel'), Neutral = neither positive કે negative action ('Remind me later').
- DatePickerDialog — Date select કરવા.
- TimePickerDialog — Time select કરવા.
- ProgressDialog — deprecated, હવે ProgressBar વપરાય.
- Custom Dialog — Custom layout સાથે (setView() method દ્વારા).
- AlertDialog button ના click events handle કરવા DialogInterface.OnClickListener interface implement કરવો પડે (View.OnClickListener થી અલગ).

```

AlertDialog.Builder builder = new AlertDialog.Builder(this);
builder.setTitle("Confirm")
    .setMessage("Are you sure you want to exit?")
    .setPositiveButton("Yes", (dialog, which) -> finish())
    .setNegativeButton("No", (dialog, which) -> dialog.dismiss())
    .show();

```

🔔 Dialog + Service Warning (Advanced Scenario)

- Service અથવા background thread context માંથી AlertDialog show કરવાનો પ્રયત્ન કરવાથી BadTokenException સાથે app crash થાય.
- કારણ: Dialogs ને attach થવા UI Activity Context (window token) જોઈએ. Services પાસે સામાન્ય રીતે window હોતું નથી.

📖 રિવિઝન પોઇન્ટ્સ

- Toast → user action ની જરૂર નથી, auto-dismiss થાય.
- Menu ના 3 types: Options, Context, Popup.
- AlertDialog → Positive, Negative, Neutral — 3 buttons સુધી હોય શકે.
- Snackbar interactive action સાથે આવે, Toast ફક્ત informational.

ભાગ ૮: List અને Adapters

8.1 ListView

- ListView એ vertically scrollable list of items બતાવે છે.
- Data ને display કરવા Adapter જરૂરી છે — Adapter, ListView (UI component) અને Data source વચ્ચે bridge તરીકે કામ કરે છે.

8.2 Adapter Types

Adapter	ઉપયોગ
ArrayAdapter	Simple array/list of strings/objects માટે. Default રીતે દરેક object પર toString() call કરી text TextView માં મૂકે
BaseAdapter	Custom layout સાથે full control માટે. તમામ Standard adapters (ArrayAdapter, CursorAdapter) નો common base/abstract class
CursorAdapter	Database (SQLite Cursor) નું data bind કરવા માટે ખાસ designed
SimpleAdapter	Static data (list of maps) bind કરવા

 **CursorAdapter નિયમ**

- Android ના legacy CursorAdapter ને primary key column ડેટાબેઝ query માં ફરજિયાત _id નામથી જ હોવો જોઈએ — row mapping આ column નામ પર આધારિત છે.

8.3 ArrayAdapter ઉદાહરણ

```
String[] items = {"Apple", "Banana", "Mango"};
ArrayAdapter<String> adapter = new ArrayAdapter<>(this,
    android.R.layout.simple_list_item_1, items);
ListView listView = findViewById(R.id.listView);
listView.setAdapter(adapter);
```

8.4 Custom Adapter (BaseAdapter) Flow

```
Data Source (ArrayList)
↓
Custom Adapter (extends BaseAdapter)
→ getView() method item layout inflate કરે
↓
ListView / RecyclerView (UI પર display)
```

```

class MyAdapter extends BaseAdapter {
    Context context;
    ArrayList<String> data;

    @Override
    public View getView(int position, View convertView, ViewGroup parent) {
        LayoutInflater inflater = LayoutInflater.from(context);
        View row = inflater.inflate(R.layout.list_item, parent, false);
        TextView tv = row.findViewById(R.id.tvItem);
        tv.setText(data.get(position));
        return row;
    }
    // getCount(), getItem(), getItemId() methods પણ override કરવા જરૂરી
}

```

- getView() method — ListView/BaseAdapter માં દરેક row બનાવવા વપરાય.
- ViewHolder pattern વગર (convertView == null ચેક કર્યા વગર) — દરેક row scroll થાય ત્યારે નવો View object બને, જેનાથી excessive object creation અને garbage collection ના કારણે scrolling jerky/laggy બને.

8.5 Item Click Listener

```

listView.setOnItemClickListener((parent, view, position, id) -> {
    Toast.makeText(this, "Clicked: " + items[position], Toast.LENGTH_SHORT).show();
});

```

સિવિઝન પોઇન્ટ્સ

- Adapter, data source અને UI component (ListView/RecyclerView) વચ્ચે bridge તરીકે કામ કરે.
- getView() method ListView/BaseAdapter માં દરેક row બનાવવા વપરાય.
- ArrayAdapter simple data માટે, BaseAdapter/CursorAdapter complex data માટે વપરાય.
- RecyclerView નો ViewHolder pattern ફરજિયાત છે, જ્યારે ListView માં developer એ manually implement કરવો પડે.

ભાગ ૯: Working with Database (SQLite)

9.1 SQLite શું છે?

- Android built-in lightweight, relational database engine છે.
- Server-less, zero-configuration, single file-based database — device પર locally store થાય.
- android.database.sqlite package દ્વારા access થાય.

9.2 SQLiteOpenHelper Class

- Database create/manage કરવા SQLiteOpenHelper class extend કરવો પડે.
- onCreate(): Database પ્રથમ વખત બને ત્યારે call થાય — અહીં table creation queries execute કરાય.
- onUpgrade(): Database upgrade કરવાની જરૂર પડે ત્યારે call થાય (દા.ત. નવો column ઉમેરવો) — database version increment કરવાથી આ trigger થાય.
- onCreate() અને onUpgrade() — SQLiteOpenHelper ના 2 mandatory (ફરજિયાત) methods છે.

```
public class DBHelper extends SQLiteOpenHelper {
    private static final String DB_NAME = "MyDatabase.db";
    private static final int DB_VERSION = 1;

    public DBHelper(Context context) {
        super(context, DB_NAME, null, DB_VERSION);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE Student(id INTEGER PRIMARY KEY, name TEXT, marks INTEGER)");
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS Student");
        onCreate(db);
    }
}
```

🔗 Mnemonic — SQLiteOpenHelper Core Methods: "C.U.D."

- C → onCreate()
- U → onUpgrade()
- D → onDowngrade()

9.3 SQLiteDatabase અને CRUD Operations

- SQLiteDatabase — CRUD operations ના methods ધરાવતો class: insert(), query()/rawQuery(), update(), delete().
- getWritableDatabase() — write operations માટે; return type SQLiteDatabase છે.
- getReadableDatabase() — read operations માટે.
- ContentValues — Insert/Update માટે વપરાતો key-value map.
- Cursor — Query result set તરફ pointer; result set પર read-write access પૂરું પાડે.

Insert

```
SQLiteDatabase db = dbHelper.getWritableDatabase();
ContentValues cv = new ContentValues();
cv.put("name", "Rahul");
cv.put("marks", 85);
db.insert("Student", null, cv);
```

Read (Query)

```
Cursor cursor = db.rawQuery("SELECT * FROM Student", null);
while (cursor.moveToNext()) {
    String name = cursor.getString(cursor.getColumnIndex("name"));
}
cursor.close(); // હંમેશા cursor close કરો
```

Update

```
ContentValues cv = new ContentValues();
cv.put("marks", 90);
db.update("Student", cv, "id=?", new String[]{"1"});
```

Delete

```
db.delete("Student", "id=?", new String[]{"1"});
```

9.4 Database Workflow

```
App → SQLiteOpenHelper → onCreate()/onUpgrade()
    ↓
    SQLiteDatabase object
    ↓
    ┌──────────┴──────────┐
    │ insert() │ update() │ delete() │ rawQuery() │
    └──────────┴──────────┘
    ↓
    Cursor (result rows)
```

9.5 SQL Injection Protection — Selection Arguments

? placeholder database query માં literal string ને directly concatenate કરવાના બદલે વપરાય છે:

```
db.rawQuery("SELECT * FROM users WHERE name = ?", new String[]{"Raj"});
```

? નો ઉપયોગ input ને literal data તરીકે treat કરાવે છે, જેથી malicious users executable SQL commands inject ન કરી શકે (SQL Injection attack અટકે).

- ContentValues vs execSQL(): DDL operations (CREATE TABLE, DROP TABLE) માટે execSQL() વાપરો. Real user data insert કરવા માટે ContentValues સાથે .insert() method વાપરો — જેથી SQL Injection vulnerabilities અટકે.

9.6 Content Provider

- Content Provider database/data ને other apps સાથે share કરવા standard interface પૂરો પાડે.
- ઉદાહરણ: Contacts, MediaStore.

9.7 Cursor સંબંધિત મહત્વના નિયમો

- The Cursor Golden Rule: નવા returned Cursor માંથી data extract કરતાં પહેલાં હંમેશા cursor.moveToFirst() call કરો. Examiners ઘણીવાર pseudo-code MCQs બનાવે છે જ્યાં આ step missing હોય, જેથી તમે CursorIndexOutOfBoundsException પકડી શકો કે નહિ તે ચકાસાય.
- Cursor.moveToFirst() return type: boolean (move successful હોય તો True, empty હોય તો False).
- Cursors ને finally block અથવા try-with-resources માં close કરવા જરૂરી — કારણ કે Cursor CursorWindow (shared memory) સાથે link ધરાવે છે; close ન કરવાથી memory occupied રહે અને app crash થાય.

9.8 આધુનિક Trends — Room, DAO, Entity, SQLCipher

Keyword	વ્યાખ્યા
Room	Jetpack architecture component જે SQLite પર abstraction layer પૂરો પાડે છે — compile-time verification of SQL queries, runtime crashes ઓછા કરે
DAO (Data Access Object)	Java/Kotlin methods ને SQL queries સાથે map કરતું interface
Entity	Room database માં SQLite table represent કરવા annotate કરેલ class
SQLCipher	SQLite માટે transparent 256-bit AES encryption પૂરું પાડતું extension
BLOB	Binary Large Object; raw bytes (દા.ત. નાની images) store કરવા SQLite data type

📝 સિવિલિયન પોઇન્ટ્સ

- SQLiteOpenHelper ના 2 mandatory methods: onCreate(), onUpgrade().
- getWritableDatabase() vs getReadableDatabase() — write vs read માટે.

- Cursor — query result iterate કરવા વપરાય, હંમેશા close કરવો.
- Content Provider → cross-app data sharing માટે.

ભાગ ૧૦: સંપૂર્ણ ટેકનિકલ કીવર્ડ બેંક

નીચે PDF ના 20 high-priority keywords + 50 advanced keywords ને એકસાથે merge કરી Gujarati વ્યાખ્યા સાથે rapid-revision glossary આપી છે — MCQ solving માટે.

કીવર્ડ સેટ ૧ — Android OS & Architecture (20 શબ્દો)

Keyword	વ્યાખ્યા / Exam Focus
ADB	Android Debug Bridge; device સાથે communicate કરવાનું command-line tool
ANR	Application Not Responding; main UI thread 5 second થી વધુ block થાય ત્યારે trigger થાય
APK	Android Package Kit; Android apps distribute/install કરવા વપરાતું file format
ART	Android Runtime; Dalvik ને replace કરી Executable (DEX) files run કરે, AOT compilation વાપરે
Context	App environment વિષે global information નું interface
Dalvik	જૂના Android versions (pre-5.0) માં વપરાતું discontinued process virtual machine
dp / dip	Density-independent pixel; UI dimensions માટે standard unit
Emulator	Computer પર Android device ને mimic કરી testing માટે વપરાતું software
Fragment	Activity નું modular, reusable, પોતાના lifecycle સાથેનું section
Gradle	Android Studio વાપરતું advanced build toolkit જે APK compile કરે
Intent	બીજા app component પાસેથી action request કરવા વપરાતો messaging object
Intent Filter	Manifest માં activity/service/broadcast receiver ની capabilities જાહેર કરે
Logcat	Android Studio નું tool જે system messages અને app debug output નો log આપે
Manifest (.xml)	AndroidManifest.xml; app package name, components, permissions ધરાવતું essential file
NDK	Native Development Kit; C અને C++ વાપરી app નો ભાગ લખવા મંજૂરી આપે
sp	Scale-independent pixels; Text sizes માટે, user ના font settings respect કરે
SQLite	Android માં embedded, open-source, lightweight relational database

Keyword	વ્યાખ્યા / Exam Focus
Toast	Screen પર દેખાતો નાનો, temporary feedback popup message
ViewTree	Layout માં Views અને ViewGroups નું hierarchical structure
Zygote	Core libraries preload કરી fast app launching enable કરતી core Android process

કીવર્ડ શેટ ૨ — Advanced UI, Adapters & Database (50 શબ્દો)

Keyword	વ્યાખ્યા / Exam Focus
Toast	Temporary pop-up message જે UI interaction ને block કરતું નથી
LENGTH_SHORT	Toast duration constant (~2 seconds)
LENGTH_LONG	Toast duration constant (~3.5 seconds)
Snackbar	Interactive action button (દા.ત. 'UNDO') સાથે આવતું modern feedback UI
Options Menu	Toolbar/app bar માં દેખાતું primary menu
Context Menu	Long-click દ્વારા ખૂલતું floating menu
Popup Menu	View ની નીચે anchor થયેલ items ની list
AlertDialog	Positive/Negative/Neutral buttons સાથે decision લેવા વપરાતું small window
DialogInterface.OnClickListener	Dialog button clicks handle કરવા વપરાતું interface
Adapter	UI component (ListView/RecyclerView) અને data source વચ્ચેનો bridge
ArrayAdapter	Array/List of objects માટે default adapter, toString() વાપરે
BaseAdapter	તમામ adapters નો common abstract base class
CursorAdapter	SQLite Cursor નું data bind કરવા વપરાતો adapter, _id column જરૂરી
ListView	Vertically scrollable list of items બતાવતું UI component
RecyclerView	ListView નું modern, ViewHolder pattern ફરજિયાત વાપરતું efficient successor
ViewHolder	findViewById() calls ઓછા કરી performance સુધારતો design pattern
LayoutManager	RecyclerView ને grid સ્વરૂપે display કરવા વપરાતો layout manager

Keyword	વ્યાખ્યા / Exam Focus
LinearLayoutManager	RecyclerView ને standard vertical/horizontal list તરીકે display કરે
StaggeredGridLayoutManager	અસમાન size ના items વાળું grid layout
SQLiteOpenHelper	Database create/version manage કરવા extend કરાતો core class
SQLiteDatabase	CRUD operations (insert, update, delete, query) ધરાવતો class
Cursor	Query result set પર pointer, read-write access આપે
ContentValues	Insert/update માટે વપરાતો key-value map
rawQuery()	Custom SQL query execute કરી Cursor return કરતી method
execSQL()	DDL operations (CREATE/DROP TABLE) execute કરવા વપરાતી method
insert()	ContentValues વાપરી નવો row ઉમેરતી method
update()	Existing rows modify કરતી method
delete()	Rows remove કરતી method
moveToFirst()	Cursor ને પ્રથમ row પર move કરે, boolean return કરે
moveToNext()	Cursor ને આગલા row પર move કરે
getColumnIndexOrThrow()	Column name ના આધારે index મેળવે, ન મળે તો exception throw કરે
onCreate() (DB)	Database પ્રથમ વખત બને ત્યારે table creation queries execute કરે
onUpgrade()	Database version વધે ત્યારે schema migrate કરવા call થાય
DB Version	Database schema ના version ને track કરતો integer
UNIQUE constraint	કોઈ column માં duplicate values ન હોય તે ensure કરે
CHECK constraint	Column value પર condition લાગુ કરે
PRIMARY KEY	દરેક row ને uniquely identify કરતો column
AUTOINCREMENT	Primary key value ને automatically વધારતું constraint
res/menu	Menu XML resources ડિફાઈન કરવાની standard ડિરેક્ટરી
res/layout	UI layout XML files ડિફાઈન કરવાની standard ડિરેક્ટરી
getWritableDatabase()	Write operations માટે SQLiteDatabase instance આપતી method

Keyword	વ્યાખ્યા / Exam Focus
getReadableDatabase()	Read operations માટે SQLiteDatabase instance આપતી method
BLOB	Raw bytes (દા.ત. તાની images) store કરવાનો SQLite data type
Room	SQLite પર compile-time verified abstraction layer પૂરો પાડતો Jetpack component
DAO	Data Access Object — Java/Kotlin methods ને SQL queries સાથે map કરે
Entity	SQLite table ને represent કરતો annotated Room class
SQLCipher	SQLite માટે 256-bit AES encryption પૂરું પાડતું extension
Context (usage)	Toast, Dialog, SQLiteOpenHelper બનાવવા જરૂરી parameter
NullColumnHack	insert() માં empty row ની મંજૂરી આપતો parameter
getView()	Adapter માં દરેક row/item નું layout inflate કરતી method

ભાગ ૧૧: વર્તમાન પ્રવાહો — Gujarat Police Context

Paper setters જ્યારે applied knowledge test કરવા માંગે, ત્યારે તેઓ recent tech implementations માંથી પ્રશ્નો pull કરે છે:

- Android Enterprise & MDM: Mobile Device Management નો Gujarat Police દ્વારા active ઉપયોગ થાય છે, જેથી official devices લોક-ડાઉન રહે અને ફક્ત approved applications જ run થાય.
- Biometric API Integration: નવા Android versions standardized BiometricPrompt APIs પૂરી પાડે છે, જેનો police mobile terminals પર fingerprint scanners ને criminal databases જેમ કે e-GujCop સાથે integrate કરવા ઉપયોગ થાય છે.
- 5G અને Edge Computing: 5G માટે Android OS optimizations, PCR વાનોને near-zero latency સાથે live dashcam અને bodycam footage State Control Room ને stream કરવા મંજૂરી આપે છે.
- Project Treble: Android નું major re-architecture, જે OS framework ને device-specific vendor software થી separate કરે છે — જેનાથી police IT departments device security patches વધુ ઝડપથી update કરી શકે.
- Room Database over Raw SQLite: Industry (અને standard IT exams) હવે Room Persistence Library તરફ shift થઈ રહી છે. Room, SQL queries નું compile-time verification પૂરું પાડે છે, જેનાથી runtime crashes ઓછા થાય. Room ને SQLite પર 'abstraction layer' તરીકે ઓળખવા સંબંધિત પ્રશ્નો expect રાખો.
- Database Encryption (SQLCipher): Sensitive police data (informant details, active warrants) ધરાવતી apps માટે, raw SQLite insecure છે કારણ કે rooted devices પર data plaintext માં store થાય છે. Gujarat state technical framework SQLCipher (256-bit AES encryption) વાપરી databases secure કરવા પર ભાર મૂકે છે.
- RecyclerView Dominance: ListView હવે largely legacy component ગણાય છે. RecyclerView modern standard છે — તેનો forced ViewHolder pattern, હજારો FIR entries ધરાવતી lists ને device lag કર્યા વગર smoothly scroll કરવા સક્ષમ બનાવે છે.

ભાગ ૧૨: ફોર્મ્યુલા અને સિન્ટેક્સ શીટ

Android OS mathematical નથી, પણ GPRB Technical papers ઘણીવાર conversions અથવા standard syntax પૂછે છે. નીચે cheat sheet છે:

1. dp (Density Independent Pixels) → px (Pixel) Conversion

```
px = dp × (dpi / 160)
(ચાદ રાખો: 160 dpi પર, 1 dp બરાબર 1 px)
```

2. DPI Calculation ફોર્મ્યુલા

```
DPI =  $\sqrt{(\text{Width}^2 + \text{Height}^2)}$  / Diagonal Size (inches)
```

ઉદાહરણ: 1080 × 1920 pixels resolution, 5 inch diagonal screen → $1080^2 = 1,166,400$; $1920^2 = 3,686,400$; સરવાળો = 4,852,800; $\sqrt{4,852,800} \approx 2202.9$ pixels; DPI = $2202.9 / 5 \approx 440.58$ dpi (xxhdpi ની નજીક).

3. Standard Minimum Touch Target Size

- 48dp × 48dp — usability ensure કરવા highly tested UI design standard.

4. Scale Factor ઉદાહરણ (dp → px on xxhdpi)

xxhdpi device (Scale factor = 3.0) પર 100dp width image → physical pixels = $100 \times 3.0 = 300$ px.

5. Implicit Intent Structure

```
Intent intent = new Intent(Intent.ACTION_VIEW);
intent.setData(Uri.parse("http://www.gujaratpolice.gujarat.gov.in"));
startActivity(intent);
```

6. Toast Message Syntax

```
Toast.makeText(context, "Text", Toast.LENGTH_SHORT).show();
```

(ધ્યાન રાખો: અંતે .show() — examiners MCQ માં ઇરાદાપૂર્વક એ છોડી દે છે એ ચકાસવા કે તમે notice કરો છો કે નહિ!)

7. Standard SQLite Table Creation

```
String CREATE_TABLE = "CREATE TABLE " + TABLE_NAME
    + "(" + COLUMN_ID + " INTEGER PRIMARY KEY AUTOINCREMENT,"
    + COLUMN_NAME + " TEXT" + ")";
db.execSQL(CREATE_TABLE);
```

8. Memory Size Calculation ઉદાહરણો

પ્રશ્ન	ગણતરી અને જવાબ
10,000 integers, દરેક 16 bytes લે તો કુલ memory કેટલી?	$10,000 \times 16 \text{ bytes} = 160,000 \text{ bytes} = 160 \text{ KB}$
100×100 pixel BitMap, ARGB_8888 (4 bytes/pixel) → memory?	$10,000 \text{ pixels} \times 4 \text{ bytes} = 40,000 \text{ bytes} \approx 39 \text{ KB} (\sim 40 \text{ KB})$
2 MB CursorWindow limit, દરેક row 500 KB BLOB → મહત્તમ rows?	$2048 \text{ KB} / 500 \text{ KB} = 4.096 \rightarrow \text{મહત્તમ 4 પૂર્ણ rows}$
400ms database query, 60Hz screen (16.6ms/frame) → skipped frames?	$400 / 16.67 \approx 24 \text{ frames skip થાય (visible freeze)}$

ભાગ ૧૩: પરીક્ષા હેઠર ટિપ્સ (GPRB Specific)

ટિપ	વિગત
Golden Rule of Lifecycles	onPause() અને onStop() વચ્ચે confuse ન થાવ. User ભાગ 'જોઈ' શકે પણ interact ન કરી શકે = Paused. બિલકુલ 'જોઈ' ન શકે = Stopped.
Architecture Mnemonic	"Little Hardware Requires Fast Apps" → Linux Kernel, HAL, Runtime/Libraries, Framework, Apps
Lifecycle Mnemonic	"Cops Start Running Past Suspect's Door" → Create, Start, Resume, Pause, Stop, Destroy
SQLiteOpenHelper Mnemonic	"C.U.D." → onCreate(), onUpgrade(), onDowngrade()
Elimination Strategy	'App components' પ્રશ્નમાં હંમેશા ચાર જ options સાચા હોય (Activity, Service, Broadcast Receiver, Content Provider); Intent/Fragment distractor છે
Cursor Golden Rule	cursor.moveToFirst() વગર data extract કરવાનો પ્રયત્ન CursorIndexOutOfBoundsException આપે
The Toast Trap	.show() missing હોય તો Toast ક્યારેય દેખાશે નહિ — 'Nothing happens' જવાબ
ContentValues vs execSQL	DDL (CREATE/DROP TABLE) માટે execSQL(); user data insert માટે ContentValues + insert() (SQL Injection સામે સુરક્ષા)
Density Pixel Fact	160 dpi પર બરાબર 1 dp = 1 px

ભાગ ૧૪: પ્રેક્ટિસ MCQ સેટ — સંપૂર્ણ સમજૂતી સાથે

Marking Scheme: સાચો જવાબ = +1, ખોટો જવાબ = -0.25, Not Attempted (Option E) = 0.

Level 1: Factual — Android OS Concepts & Architecture

Q1. Android એ કયા kernel પર based છે?

- A) Windows Kernel
- B) Linux Kernel
- C) Mac Kernel
- D) Unix Kernel
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: Android એ Linux-kernel based, open-source operating system છે.

Q2. Android Architecture માં કેટલા layers છે?

- A) 3
- B) 4
- C) 5
- D) 7
- E) Not Attempted

✓ જવાબ: C

સમજૂતી: Linux Kernel, HAL, Libraries+ART, Application Framework, Applications — કુલ 5 layers.

Q3. Dalvik VM ને replace કરનાર runtime કયું છે?

- A) JVM
- B) ART
- C) CLR
- D) NDK
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: ART (Android Runtime) એ Android 5.0 Lollipop થી Dalvik ને replace કર્યું, AOT compilation વાપરે.

Q4. Google એ Android માટે 2019 માં કયું language preferred (first-class) જાહેર કર્યું?

- A) Java
- B) C++
- C) Kotlin
- D) Python
- E) Not Attempted

✓ જવાબ: C

સમજૂતી: Java હજુ supported છે, પણ 2019 થી Kotlin preferred language છે.

Q5. R.java ફાઇલમાં શું હોય છે?

- A) App નો source code
- B) દરેક resource (layouts, strings, images) માટે Resource IDs
- C) Database schema
- D) Manifest configurations

E) Not Attempted

✓ જવાબ: B

સમજૂતી: R.java build process દ્વારા auto-generate થાય છે અને res folder ના દરેક resource ને unique integer ID આપે.

Level 2: Application — Architecture, Activity, Intent

Q6. Device camera hardware access કરવા Android Architecture નું કયું layer Java framework ને capability expose કરે?

- A) System Apps
- B) Hardware Abstraction Layer (HAL)
- C) Native Libraries
- D) Linux Kernel
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: HAL, hardware vendors માટે standard interface implement કરતા library modules ધરાવે છે.

Q7. Android એ Dalvik (JIT) થી ART (AOT) પર switch શા માટે કર્યું?

- A) Storage બચાવવા
- B) Installation સમયે compile કરી app launch speed અને battery performance સુધારવા
- C) C++ support કરવા
- D) Linux Kernel remove કરવા
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: AOT compilation, installation સમયે bytecode ને native code માં compile કરી execution time overhead ઘટાડે.

Q8. 2D અને 3D graphics rendering માટે કઈ native library જવાબદાર છે?

- A) SQLite
- B) OpenGL / Vulkan
- C) WebKit
- D) SSL
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: OpenGL ES (અને હવે Vulkan) Android framework ના standard graphics libraries છે.

Q9. App ની અંદર lightweight relational database માટે કઈ library વપરાય?

- A) MySQL
- B) OracleDB
- C) SQLite
- D) MongoDB
- E) Not Attempted

✓ જવાબ: C

સમજૂતી: SQLite એ Android native libraries માં આપેલ standard embedded SQL database engine છે.

Q10. Activity Manager Android Architecture ના કયા layer નો ભાગ છે?

- A) Linux Kernel
- B) Java API Framework
- C) System Runtime

- D) Hardware Abstraction Layer
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: Java API Framework Activity Manager, Notification Manager જેવા high-level Java classes પૂરા પાડે.

Level 3: Advanced — Activity Lifecycle & Intent

Q11. Activity running છે, user ને phone call આવે, Activity background માં જાય પણ destroy નથી થતી. કયો lifecycle method call થાય?

- A) onStop()
- B) onPause()
- C) onDestroy()
- D) onRestart()
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: onPause() ત્યારે call થાય જ્યારે activity foreground focus ગુમાવે (partially visible/about to stop). સંપૂર્ણ covered હોય તો onStop() follow થાય.

Q12. Activity બંને ત્યારે, UI initialize (setContentView) કરવા સૌથી પહેલા કયો method call થાય?

- A) onStart()
- B) onResume()
- C) onCreate()
- D) onInit()
- E) Not Attempted

✓ જવાબ: C

સમજૂતી: onCreate() entry point છે જ્યાં activity initialize થાય અને UI layout inflate થાય.

Q13. Implicit Intent નો હેતુ શું છે?

- A) ચોક્કસ activity ને class name થી start કરવો
- B) Variable declare કરવો
- C) 'View Map' કે 'Send Email' જેવી action request કરવી, ચોક્કસ app specify કર્યા વગર
- D) Service stop કરવો
- E) Not Attempted

✓ જવાબ: C

સમજૂતી: Implicit intents બીજી apps ના components ને general action handle કરવા મંજૂરી આપે.

Q14. બે Activities વચ્ચે data pass કરવા primarily શું વપરાય?

- A) Bundle (Intent દ્વારા)
- B) Toast
- C) SharedPreferences
- D) Manifest
- E) Not Attempted

✓ જવાબ: A

સમજૂતી: Intent, putExtra() વાપરી key-value data નું Bundle carry કરી શકે.

Q15. User Screen rotate કરે, Activity destroy અને recreate થાય. User ની scroll position કેવી રીતે save કરવી?

- A) Text file માં write કરવું
- B) onSaveInstanceState() વાપરી Bundle માં data store કરવો
- C) Save કરવું અશક્ય છે
- D) અલગ SQL database વાપરવો
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: onSaveInstanceState() transient UI state (જેમ કે scroll position) activity kill થતાં પહેલાં save કરવા designed છે.

Q16. Back stack clear કરી fresh task શરૂ કરવા કયો Intent flag વપરાય?

- A) FLAG_ACTIVITY_NEW_TASK
- B) FLAG_ACTIVITY_CLEAR_TOP
- C) FLAG_ACTIVITY_SINGLE_TOP
- D) A અને B બંને (ઘણીવાર સાથે વપરાય)
- E) Not Attempted

✓ જવાબ: D

સમજૂતી: FLAG_ACTIVITY_NEW_TASK | FLAG_ACTIVITY_CLEAR_TASK (અથવા CLEAR_TOP) history wipe કરી fresh start માટે વપરાય (દા.ત. Logout પછી).

Q17. ANR (Application Not Responding) error શાના કારણે થાય?

- A) Low battery
- B) Main UI Thread પર long-running operations (જેમ કે network requests) perform કરવાથી
- C) Layout માં ઘણી images
- D) Manifest permission missing
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: Main Thread UI updates માટે જવાબદાર છે. તેને 5 second થી વધુ block કરવાથી ANR trigger થાય.

Q18. User Back button દબાવી app exit કરે ત્યારે lifecycle calls નો સાચો sequence કયો?

- A) onPause() → onStop() → onDestroy()
- B) onDestroy() → onStop()
- C) onStop() → onPause()
- D) onPause() → onResume()
- E) Not Attempted

✓ જવાબ: A

સમજૂતી: Activity focus ગુમાવે (onPause), invisible બને (onStop), અને છેલ્લે memory માંથી remove થાય (onDestroy).

Q19. Entire system માં Activity ના ફક્ત એક જ instance, પોતાના અલગ task માં ensure કરવા કયો launch mode વપરાય?

- A) standard
- B) singleTop
- C) singleTask
- D) singleInstance
- E) Not Attempted

✓ જવાબ: D

સમજૂતી: singleInstance સૌથી restrictive mode છે; activity પોતાની task માં run થાય, બીજી કોઈ activity એ task માં launch ન થઈ શકે.

Level 4: Technical Calculations — UI Design

Q20. Device screen resolution 1080 × 1920 pixels, diagonal size 5 inches. DPI ગણો.

- A) ~300 dpi
- B) ~440 dpi
- C) ~200 dpi
- D) ~600 dpi
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: $\sqrt{(1080^2 + 1920^2)} = \sqrt{4,852,800} \approx 2202.9 \text{ pixels}$; $DPI = 2202.9 / 5 \approx 440.58 \text{ dpi}$.

Q21. ImageView 100dp width સેટ છે. xxhdpi device (Scale factor = 3.0) પર કેટલા physical pixels occupy કરશે?

- A) 100 px
- B) 200 px
- C) 300 px
- D) 400 px
- E) Not Attempted

✓ જવાબ: C

સમજૂતી: $100dp \times 3.0 \text{ scale factor} = 300 \text{ physical pixels}$.

Level 5: Advanced UI — Toast, Menu, Dialog, RecyclerView

Q22. Toast ને actually screen પર display કરવા કયો method call ફરજિયાત છે?

- A) display()
- B) show()
- C) visible()
- D) render()
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: `Toast.makeText(...).show()` ફરજિયાત chain છે. `.show()` વગર toast create તો થાય પણ ક્યારેય દેખાય નહિ.

Q23. Android Project structure માં Menu resources (XML files) ડિફાઈન કરવાની standard location કઈ છે?

- A) res/layout
- B) res/values
- C) res/menu
- D) res/drawable
- E) Not Attempted

✓ જવાબ: C

સમજૂતી: Android Studio menu XML definitions (દા.ત. `main_menu.xml`) ને `res/menu` ડિરેક્ટરીમાં expect કરે.

Q24. Android માં તમામ standard adapters (જેમ કે ArrayAdapter, CursorAdapter) નો base class કયો છે?

- A) ViewAdapter

- B) BaseAdapter
- C) ListAdapter
- D) RecyclerViewAdapter
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: BaseAdapter એ Adapter interface implement કરતો common abstract class છે.

Q25. RecyclerView માં ViewHolder નો primary purpose શું છે?

- A) Database connection hold કરવો
- B) Views (TextView, ImageView) ના references cache કરી findViewById calls ઓછા કરવા
- C) Data sort કરવો
- D) ફક્ત click events handle કરવા
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: ViewHolder pattern, દરેક વખતે row scroll થાય ત્યારે expensive findViewById() call થતું અટકાવે.

Q26. 'Long-click' થી ખૂલતું floating menu કયું છે?

- A) Options Menu
- B) Context Menu
- C) Popup Menu
- D) Navigation Menu
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: Context Menu, item-specific actions (દા.ત. lists) માટે long-press થી ખૂલે.

Q27. AlertDialog.Builder ની કઈ method 'Cancel' button ઉમેરે જે dialog ને કોઈ action લીધા વગર બંધ કરે?

- A) setPositiveButton
- B) setNegativeButton
- C) setNeutralButton
- D) setExitButton
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: Convention પ્રમાણે setNegativeButton, 'Cancel'/'No' અથવા destructive dismissal actions માટે વપરાય.

Q28. Service અથવા background thread context માંથી AlertDialog show કરવાનો પ્રયત્ન કરવાથી શું થાય?

- A) Dialog normal રીતે દેખાય
- B) App BadTokenException સાથે crash થાય
- C) Database lock થાય
- D) Nothing happens
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: Dialogs ને attach થવા UI Activity Context (window token) જોઈએ, જે Services પાસે સામાન્ય રીતે નથી હોતું.

Level 6: Database (SQLite) — Factual & Application

Q29. getWritableDatabase() method નો return type શું છે?

- A) SQLiteOpenHelper
- B) SQLiteDatabase
- C) Cursor
- D) boolean
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: તે SQLiteDatabase નું instance return કરે છે જે insert, update, delete operations ની methods પૂરા પાડે.

Q30. SQLiteOpenHelper માં database version 1 થી 2 change કરો છો. System automatically કયો method call કરશે?

- A) onCreate
- B) onDowngrade
- C) onUpgrade
- D) onUpdate
- E) Not Attempted

✓ જવાબ: C

સમજૂતી: Helper stored version vs requested version ચેક કરે છે. Requested > stored હોય તો onUpgrade() trigger થાય schema migration માટે.

Q31. Cursor objects ને finally block/try-with-resources માં કેમ close કરવા જોઈએ?

- A) ફક્ત battery બચાવવા
- B) Memory leaks અટકાવવા અને database layer ની connection free કરવા
- C) Data sort કરવા
- D) Transaction commit કરવા
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: Cursors CursorWindow (shared memory) સાથે link ધરાવે છે. Close ન કરવાથી memory occupied રહી app crash થાય.

Q32. SQL command જે table ને database માંથી સંપૂર્ણપણે remove કરે?

- A) DELETE TABLE
- B) REMOVE TABLE
- C) DROP TABLE
- D) ERASE TABLE
- E) Not Attempted

✓ જવાબ: C

સમજૂતી: DROP TABLE standard SQL command છે table structure અને data બંને delete કરવા.

Q33..rawQuery() માં 'name = ?' જેવો query લખ્યો છો, અને directly string ના બદલે '?' કેમ વાપર્યો છો?

- A) Code ટૂંકો બનાવવા
- B) SQL Injection attacks અટકાવવા
- C) SQL strings support નથી કરતું
- D) એ table name નું placeholder છે
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: Selection arguments (?) input ને literal data તરીકે treat કરાવે, malicious SQL injection અટકાવે.

Q34. Email ID જેવા column માં duplicate values ન હોય તે કઈ SQL constraint ensure કરે?

- A) CHECK
- B) UNIQUE
- C) DISTINCT
- D) INDEX
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: UNIQUE constraint બે rows ને specified column માં સમાન value રાખતી અટકાવે.

Q35. SQLite database ની page size 4 KB છે. તમે 2 KB ના rows insert કરો છો. એક page માં કેટલા rows સમાય?

- A) 1
- B) 2
- C) 4
- D) 0.5
- E) Not Attempted

✓ જવાબ: B

સમજૂતી: $4\text{ KB} / 2\text{ KB} = 2\text{ rows per page}$.

ભાગ ૧૫: ટૂંકા જવાબી પ્રશ્નો (Short Answer Type)

- 1. Android Architecture ના 5 layers ને ટૂંકમાં સમજાવો.
- 2. Activity Lifecycle ના methods diagram સાથે વર્ણવો.
- 3. Explicit Intent અને Implicit Intent વચ્ચે તફાવત આપો.
- 4. Toast, Dialog અને Menu વચ્ચે તફાવત આપો.
- 5. ArrayAdapter અને BaseAdapter વચ્ચે તફાવત આપો.
- 6. SQLite database માં CRUD operations ને code સાથે સમજાવો.
- 7. Fragment અને Activity વચ્ચે તફાવત આપો.
- 8. ConstraintLayout ના ફાયદા જણાવો.
- 9. ART, Dalvik VM ને કેમ replace કરે છે તે સમજાવો.
- 10. RecyclerView, ListView કરતાં કેમ વધુ efficient છે તે ViewHolder pattern સાથે સમજાવો.
- 11. Room Database, Raw SQLite કરતાં કેમ preferable છે તે સમજાવો.
- 12. SQLCipher નો ઉપયોગ police data security માટે કેમ મહત્વનો છે તે સમજાવો.

ભાગ ૧૬: આઈન્ડ મેપ રિફેપ

ANDROID APPLICATION DEVELOPMENT

- 1. Android OS Concepts
 - Open-source, Linux-based, OHA, Versions, 4 Core Components
- 2. Android Architecture (5 Layers)
 - Linux Kernel → HAL → Libraries+ART → Framework → Applications
- 3. Activities & UI Design
 - Activity Lifecycle (7 methods)
 - Intent (Explicit / Implicit)
 - Layouts (Linear, Relative, Constraint) + Widgets
- 4. Advanced UI
 - Fragment
 - RecyclerView (ViewHolder)
 - Custom View
 - Animation (Tween/Property/Frame)
 - Material Design
- 5. Toast, Menu, Dialog
 - Toast → Auto-dismiss message
 - Menu → Options / Context / Popup
 - Dialog → AlertDialog, DatePicker, TimePicker
- 6. List & Adapters
 - ListView + Adapter (bridge)
 - ArrayAdapter / BaseAdapter / CursorAdapter
- 7. Database (SQLite)
 - SQLiteOpenHelper (onCreate, onUpgrade)
 - CRUD: insert(), update(), delete(), rawQuery()
 - Room / DAO / Entity / SQLCipher (Advanced Trends)
 - Content Provider (data sharing)

🔗 અંતિમ Exam Tip

- Activity Lifecycle, Android Architecture Layers, અને Adapter types — આ ત્રણ topics પર PSI exam માં વારંવાર direct MCQ પુછાય છે. Diagrams અને mnemonics સાથે યાદ રાખો.
- GPRB Technical calculations (DPI, memory size, frame drops) માટે formula sheet ને revision પહેલાં એકવાર ફરીથી જુઓ.
- Golden Rules (Cursor, Lifecycle, Toast Trap) — MCQ ના distractor options ને પકડવા આ rules ખૂબ ઉપયોગી છે.